

Relatório de implementação Compilador da Linguagem Homi

Guilherme M. Einloft and Gabriel Stiegemeier

Ciência da Computação

Universidade Federal de Santa Maria

Santa Maria, RS, Brazil 97105-900

Email: gmeinloft@inf.ufsm.br and gstiegemeier@inf.ufsm.br

I. INTRODUÇÃO E VISÃO GERAL

O presente relatório descreve o projeto e a implementação de um compilador modular para a linguagem Homi, desenvolvido em Python 3 com auxílio da biblioteca PLY (Python Lex-Yacc). A linguagem Homi foi concebida para permitir que usuários leigos descrevam automações residenciais de forma procedural e intuitiva, utilizando palavras-chave em português. O compilador traduz esses scripts para arquivos declarativos YAML compatíveis com o sistema de automação residencial Home Assistant.

A. Arquitetura do pipeline

O compilador segue a arquitetura clássica de fases, organizada em quatro módulos sequenciais, como exemplificado na Figura 1.

O ponto de entrada é o arquivo `main.py`, que orquestra a execução sequencial de todas as fases, lendo um arquivo `.homi` de entrada e produzindo um arquivo `.yaml` de saída:

```
# Pipeline de compilacao em main.py
# (simplificado):
# Fases 1 e 2
ast = parser.parse(codigo_fonte, lexer=lexer)
# Fase 3
sem_erros = AnalisadorSemantico().analisar(ast)
# Fase 4
GeradorYAML(ast).salvar_arquivo(arquivo_saida)
```

II. DESCRIÇÃO DA GRAMÁTICA LIVRE DE CONTEXTO

A Gramática Livre de Contexto (GLC) que define a estrutura sintática da linguagem Homi é apresentada formalmente a seguir na notação BNF (Backus-Naur Form).

A. Símbolos não-terminais

Os símbolos não-terminais representam abstrações sintáticas que são expandidas por regras de produção. Estão delimitados por `< >` e descritos na Tabela I.

B. Símbolos terminais

Os símbolos terminais são os tokens produzidos pelo analisador léxico e recebidos pelo parser, descritos na Tabela II.

TABLE I
DESCRIÇÃO DOS SÍMBOLOS NÃO-TERMINAIS DA GRAMÁTICA

Não-Terminal	Descrição
<code><programa></code>	Nó raiz: lista de uma ou mais automações
<code><automacao></code>	Estrutura completa de uma automação
<code><bloco_moda></code>	Declaração opcional do modo de execução
<code><bloco_gatilho></code>	Bloco obrigatório de gatilhos (QUANDO:)
<code><lista_gatilhos></code>	Sequência de um ou mais gatilhos
<code><com_gatilho></code>	Um gatilho individual
<code><bloco_condicao></code>	Bloco opcional de condições (SE:)
<code><lista_condicoes></code>	Sequência de uma ou mais condições
<code><com_condicao></code>	Uma condição individual
<code><bloco_acao></code>	Bloco obrigatório de ações (FAÇA:)
<code><lista_acoes></code>	Sequência de uma ou mais ações
<code><com_acao></code>	Uma ação individual

C. Produções da GLC

As produções da Gramática Livre de contexto estão descritas na Tabela III.

A produção ϵ (vazia) aparece em `<bloco_moda>` e `<bloco_condicao>`, tornando o modo de execução e o bloco de condições opcionais. Quando `<bloco_moda>` deriva ϵ , o valor padrão adotado é `'single'`.

III. ANALISADOR LÉXICO

O Analisador Léxico (`lexer.py`) constitui a primeira fase do compilador. Sua função é varrer o código-fonte caractere a caractere, da esquerda para a direita, e agrupá-los em unidades lógicas mínimas com significado, denominadas tokens. Paralelamente, descarta elementos irrelevantes para a compilação, como espaços, tabulações e comentários, e rastreia o número de linhas para reporte preciso de erros.

A implementação utiliza o módulo `ply.lex` da biblioteca PLY, que internamente constrói um Autômato Finito Determinístico (AFD) a partir das expressões regulares declaradas para cada regra de token.

A. Palavras reservadas

O primeiro componente estrutural é o dicionário `reserved`, que mapeia cada palavra-chave da linguagem ao seu respectivo tipo de token:

```
reserved = {
    'AUTOMACAO': 'AUTOMACAO',
    'MODO': 'MODO',
```

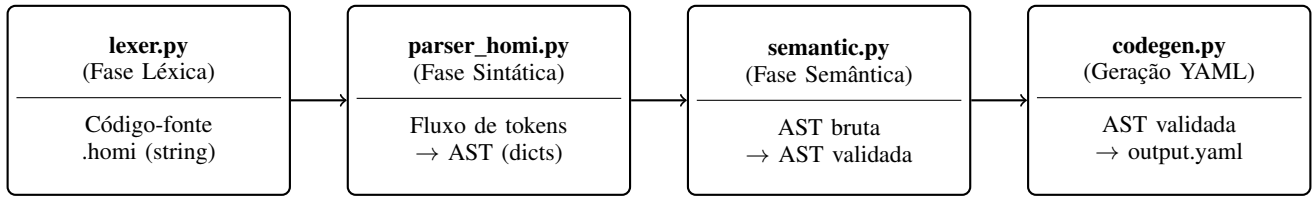


Fig. 1. Pipeline de compilação: fluxo de dados desde a análise léxica até a geração do código alvo.

TABLE II
DESCRIÇÃO DOS SÍMBOLOS TERMINAIS E TOKENS DA GRAMÁTICA

Terminal	Exemplo	Descrição
AUTOMACAO	AUTOMACAO	Início de automação
MODO	MODO	Declaração de modo
IDENTIFICADOR	single, restart	Identificador de modo
QUANDO	QUANDO	Bloco de gatilhos
SE	SE	Bloco de condições
FACA	FAÇA / FACA	Bloco de ações
FIM	FIM	Fim de automação
LIGAR	LIGAR	Ação de ativar uma entidade
DESLIGAR	DESLIGAR	Ação de desativar uma entidade
ESPERAR	ESPERAR	Ação de aguardar um tempo
ESTA	ESTA	Operador de comparação de estado
ACIMA	ACIMA	Operador de comparação numérica (>)
ABAIXO	ABAIXO	Operador de comparação numérica (<)
ENTRE	ENTRE	Operador de intervalo temporal
E	E	Conjunção para intervalos
HORARIO	HORARIO	Palavra-chave para condição/gatilho temporal
STRING	"off", "disarmed"	Literal de texto entre aspas duplas
ENTIDADE	light.sala	Identificador de entidade do Home Assistant
EVENTO	sunset, sunrise	Evento geográfico/solar
TEMPO	5min, -01:30:00	Literal de duração temporal
NUMERO	20, 75	Literal inteiro
DOISPONTOS	:	Delimitador de seção
TRACO	-	Prefixo de item de lista

TABLE III
GRAMÁTICA LIVRE DE CONTEXTO (GLC) DA LINGUAGEM EM FORMATO BNF

<pre> <programa> ::= <automacao> <programa> <automacao> <automacao> ::= AUTOMACAO STRING <bloco_modos> <bloco_gatilho> <bloco_condicao> <bloco_acao> FIM <bloco_modos> ::= MODO IDENTIFICADOR ε <bloco_gatilho> ::= QUANDO DOISPONTOS <lista_gatilhos> <lista_gatilhos> ::= <com_gatilho> <lista_gatilhos> <com_gatilho> <com_gatilho> ::= TRACO EVENTO TEMPO TRACO ENTIDADE ESTA STRING TRACO ENTIDADE ACIMA NUMERO TRACO ENTIDADE ABAIXO NUMERO TRACO HORARIO ENTRE TEMPO E TEMPO <bloco_condicao> ::= SE DOISPONTOS <lista_condicoes> ε <lista_condicoes> ::= <com_condicao> <lista_condicoes> <com_condicao> <com_condicao> ::= TRACO ENTIDADE ESTA STRING TRACO ENTIDADE ACIMA NUMERO TRACO ENTIDADE ABAIXO NUMERO TRACO HORARIO ENTRE TEMPO E TEMPO <bloco_acao> ::= FACA DOISPONTOS <lista_acoes> <lista_acoes> ::= <com_acao> <lista_acoes> <com_acao> <com_acao> ::= TRACO LIGAR ENTIDADE TRACO DESLIGAR ENTIDADE TRACO ESPERAR TEMPO </pre>	<pre> ' HORARIO' : ' HORARIO' , } </pre>
--	---

```

'QUANDO' :      'QUANDO' ,
'SE' :         'SE' ,
'FACA' :       'FACA' ,
'FAÇA' :       'FACA' ,
'LIGAR' :      'LIGAR' ,
'DESLIGAR' :   'DESLIGAR' ,
'ESPERAR' :    'ESPERAR' ,
'ESTA' :       'ESTA' ,
'FIM' :        'FIM' ,
'ACIMA' :      'ACIMA' ,
'ABAIXO' :     'ABAIXO' ,
'ENTRE' :      'ENTRE' ,
'E' :          'E' ,

```

Destaca-se que 'FAÇA' (com cedilha) e 'FACA' (sem cedilha) mapeiam ambos para o token FACA, para que o compilador aceite as duas grafias. A lista de tokens é construída combinando os tokens dinâmicos com os tokens das palavras reservadas.

B. Estrutura do Autômato Finito Determinístico

O PLY internamente compila cada expressão regular declarada em uma máquina de estados (AFD), obedecendo a uma regra de precedência rígida:

- 1) Regras definidas como funções (`t_STRING`, `t_TEMPO`, etc.) são avaliadas na ordem em que aparecem no arquivo-fonte (de cima para baixo).
- 2) Regras definidas como variáveis/strings são avaliadas por tamanho decrescente do padrão regex.

C. Reconhecimento de tokens complexos

Token ENTIDADE: Identificadores do Home Assistant no formato `domínio.nome`:

```
def t_ENTIDADE(t):
    r'[a-z][a-z0-9_]*\.[a-z0-9][a-z0-9_]*'
    return t
```

O AFD para esta regra transita por um estado inicial esperando uma letra minúscula, seguida de zero ou mais caracteres alfanuméricos ou underscores, obrigatoriamente um ponto literal (`.`), e então uma segunda sequência iniciada por letra minúscula ou dígito. Exemplos válidos: `light.sala`, `binary_sensor.1e1e192bce0cdc593`.

Token TEMPO: Múltiplos formatos temporais com alternâncias:

```
def t_TEMPO(t):
    r'-(?:\d{1,2}:\d{2}:\d{2}|
    (?:\d+h[ \t]*)?(?:\d+min[ \t]*)?'
    r'\d+s|(?:\d+h[ \t]*)?\d+min|\d+h'
    t.value = t.value.strip()
    return t
```

A expressão regular utiliza o operador de alternância (`|`) para reconhecer três formatos:

TABLE IV
FORMATOS SUPORTADOS PELO TOKEN TEMPO

Formato	Exemplo	Padrão Regex
Offset HH:MM:SS	-01:30:00	<code>-(?:\d{1,2}:\d{2}:\d{2} </code>
Composto	1h 2min, 1min 45s	<code>(?:\d+h[\t]*)?(?:\d+min[\t]*)?</code>
Isolado	5min, 10s, 2h	<code>\d+s \d+h[\t]*)?\d+min \d+h</code>

Token PALAVRA (regra genérica): Captura qualquer cadeia alfanumérica e consulta o dicionário de reservas:

```
def t_PALAVRA(t):
    r'[A-Za-z_Çç][A-Za-z0-9_Çç]*'
    t.type = reserved.get(t.value, 'IDENTIFICADOR')
    return t
```

Se a palavra estiver no dicionário `reserved`, o tipo do token é atualizado para a palavra-chave correspondente; caso contrário, se não for um EVENTO conhecido (`sunset` e `sunrise`), é classificada como `IDENTIFICADOR`. Isso garante extensibilidade, mas delega a validação de nomes desconhecidos ao parser.

D. Tratamento de comentários

Comentários iniciados por `#` são descartados silenciosamente pela seguinte regra:

```
def t_COMENTARIO(t):
    r'\#.*'
    pass # Nao retorna nada
```

A instrução `pass` faz com que o PLY consuma o comentário inteiro, sem emitir nenhum token para o parser.

E. Tratamento de erros léxicos

Quando o lexer encontra um caractere que não casa com nenhuma regra definida:

```
def t_error(t):
    print(
        f"[ERRO LEXICO] Caractere '{t.value[0]}' "
        f"na linha {t.lineno}, pos. {t.lexpos}"
    )
    t.lexer.lex_error = True
    t.lexer.skip(1)
```

A função emite uma mensagem com o caractere ilegal, a linha e a posição. A flag `lex_error` é ativada para sinalização posterior, e o caractere é pulado com `skip(1)` para que o lexer tente continuar a análise sem abortar.

IV. ANALISADOR SINTÁTICO

O Analisador Sintático (`parser_homi.py`) utiliza o módulo `ply.yacc`, que constrói um parser Bottom-Up LALR(1) (Look-Ahead LR com 1 token de lookahead). Trata-se de uma variante do algoritmo LR que utiliza tabelas de análise compactadas geradas automaticamente pelo PLY, salvas no arquivo `'parsetab.py'`.

O parser LALR(1) opera por meio de duas operações fundamentais:

- **Shift** (empilhamento): desloca o token corrente para a pilha de análise.
- **Reduce** (redução): aplica uma regra de produção da gramática, substituindo os símbolos do topo da pilha pelo não-terminal da cabeça da regra.

1) *Construção da AST:* Cada regra gramatical é declarada como uma função Python prefixada com `p_`, cuja docstring descreve formalmente a produção BNF. O argumento `p` é um objeto `YaccProduction` que se comporta como uma lista indexada:

- `p[0]`: resultado da produção (lado esquerdo da regra).
- `p[1]`, `p[2]`, ...: símbolos do lado direito da regra.

A regra principal demonstra a construção do nó de automação na AST:

```
def p_automacao(p):
    '''automacao : AUTOMACAO STRING
    bloco_modos
    bloco_gatilho
    bloco_condicao
    bloco_acao
```

```

FIM'''
p[0] = {
    'tipo': 'automacao',
    'nome': p[2],
    'modo': p[3],
    'gatilhos': p[4],
    'condicoes': p[5],
    'acoes': p[6],
}

```

A AST resultante é uma lista de dicionários Python, onde cada dicionário encapsula hierarquicamente os componentes semânticos de uma automação. Exemplo de AST:

```

[
{
  'tipo': 'automacao',
  'nome': '"Por do sol na Sala"',
  'modo': 'single',
  'gatilhos': [
    {
      'tipo': 'gatilho_evento',
      'evento':
        'sunset',
      'offset':
        '-01:30:00'
    }
  ],
  'condicoes': [
    {
      'tipo': 'condicao_estado',
      'entidade': 'alarm_control_panel.
        alarmo',
      'estado': '"disarmed"'
    },
    {
      'tipo': 'condicao_estado',
      'entidade': 'light.sala',
      'estado': '"off"'
    }
  ],
  'acoes': [
    {
      'tipo': 'acao_ligar',
      'entidade': 'light.sala'
    },
    {
      'tipo': 'acao_esperar',
      'duracao': '5min'
    },
    {
      'tipo': 'acao_desligar',
      'entidade': 'light.sala'
    }
  ]
}
]

```

A. Recuperação de erros

O parser implementa uma estratégia de recuperação de erros em Modo Pânico, permitindo que a compilação continue mesmo após encontrar erros sintáticos. Essa abordagem opera em dois níveis:

1) *Nível de automação*: Quando a estrutura interna de uma automação está inválida, o parser utiliza o token especial `error` do PLY para consumir e descartar tokens até encontrar o delimitador de sincronização `FIM`:

```

def p_automacao_erro(p):
    '''automacao : AUTOMACAO STRING
        error
        FIM'''

    print(
        f"[ERRO SINTATICO] Estrutura invalida
na automacao {p[2]}. "
        f"Recuperando no token FIM (linha
{p.lineno(4)})."
    )
    parser.syntax_error = True
    p[0] = {
        'tipo': 'automacao_com_erro',
        'nome': p[2],
        'modo': 'single',
        'gatilhos': [],
        'condicoes': [],
        'acoes': [],
    }

```

O nó gerado é rotulado com o tipo `'automacao_com_erro'`, permitindo que as fases subsequentes (semântica e geração de código) ignorem essa automação sem interromper o processamento das demais.

2) *Nível global*: A função `p_error` é chamada automaticamente pelo Yacc no primeiro token que viola a gramática. Ela implementa a sincronização consumindo tokens até encontrar um token de sincronização:

```

def p_error(p):
    if p is None:
        print("[ERRO SINTATICO] Fim inesperado
do arquivo (EOF).")
        parser.syntax_error = True
        return

    print(
        f"[ERRO SINTATICO] Token inesperado
'{p.value}' "
        f"(tipo: {p.type}) na linha {p.lineno
}."
    )
    parser.syntax_error = True

    # Tokens de sincronização
    sync_tokens = {'FIM', 'QUANDO', 'SE', '
FACA',
'AUTOMACAO'}

    while True:
        tok = p.lexer.token()
        if tok is None:
            break
        if tok.type in sync_tokens:
            parser.errok()
            parser.restart()
            _original_token = parser.token
            parser.token = lambda _tok=tok,
            _orig=_original_token: (
                setattr(parser, 'token', _orig
)

```

```

or _tok
    )
    break

```

O conjunto de tokens de sincronização (FIM, QUANDO, SE, FACA, AUTOMACAO) representa delimitadores de bloco que definem fronteiras estruturais seguras. Ao encontrar um desses tokens:

- 1) `parser.errok()` limpa a flag de estado de erro de PLY.
- 2) `parser.restart()` reinicia o estado interno do autômato LALR.
- 3) O token de sincronização é reinjetado na fila de leitura por meio de uma função lambda que substitui temporariamente o método `parser.token`, garantindo que o parser retorne a análise a partir desse ponto.

B. Analisador semântico

O Analisador Semântico (`semantic.py`) constitui a terceira fase do compilador. Enquanto as fases anteriores validam a forma (léxica) e a gramática (sintática) do código, esta fase valida a coerência lógica do programa, garantindo que as operações escritas pelo usuário façam sentido no contexto do Home Assistant.

A implementação utiliza a classe `AnalisadorSemantico`, que adota o padrão de projeto Visitor para percorrer recursivamente os nós da AST.

C. Tabela de símbolos

A Tabela de Símbolos é implementada como um dicionário Python (`self.tabela_simbolos`) que mapeia cada entidade encontrada no código-fonte ao seu respectivo domínio:

```

def __init__(self):
    self.tabela_simbolos = {}
    self.erros = []

def _registrar_entidade(self, entidade: str):
    if entidade not in self.tabela_simbolos:
        dominio = self._extrair_dominio(
            entidade)
        self.tabela_simbolos[entidade] =
            dominio

```

O domínio é extraído da parte anterior ao ponto do identificador de entidade (ex: `'light'` de `'light.sala'`). A tabela é populada progressivamente conforme o Visitor percorre os blocos de gatilhos, condições e ações.

D. Checagem de tipos e consistência externa

A validação semântica central do compilador impede que ações de controle (LIGAR/DESLIGAR) sejam aplicadas a entidades de domínios exclusivamente de leitura. Para isso, o analisador mantém dois conjuntos estáticos de classificação:

Domínios Atuadores (aceitam LIGAR/DESLIGAR):

```

DOMINIOS_ATUADORES = {
    'light', 'switch', 'fan', 'cover', 'lock',
    'media_player', 'climate', 'vacuum',
    'script', 'automation', 'input_boolean',

```

```

    'scene', 'input_select', 'timer', 'notify'
    'alexa_devices',
}

```

Domínios Sensores (somente leitura, protegidos de alteração):

```

DOMINIOS_SENSORES = {
    'sensor', 'binary_sensor', 'weather',
    'sun', 'device_tracker', 'zone',
    'person', 'alarm_control_panel',
}

```

A checagem de tipos é realizada no método `_visitar_acao`:

```

def _visitar_acao(self, no: dict,
    automacao_nome: str):
    if no['tipo'] in ('acao_ligar', '
        acao_desligar'):
        entidade = no['entidade']
        self._registrar_entidade(entidade)
        dominio = self._extrair_dominio(
            entidade)

        if dominio not in self.
            DOMINIOS_ATUADORES:
            verbo = 'LIGAR' if no['tipo'] == '
                acao_ligar' else 'DESLIGAR'
            self._adicionar_erro(
                automacao_nome,
                f"Ação '{verbo}' inválida para
                    a entidade '{entidade}'."
                )

            f"O domínio '{dominio}' é de
                leitura e não aceita "
            f"comandos de estado."
        )

```

E. Acumulação de erros

O analisador nunca aborta no primeiro erro encontrado. Todos os erros são acumulados na lista `self.erros`, permitindo que o relatório final apresente todas as inconsistências de uma só vez. Automações marcadas como `'automacao_com_erro'` pelo parser são silenciosamente ignoradas, evitando falsos positivos decorrentes de falhas sintáticas já reportadas.

V. GERAÇÃO DE CÓDIGO

O Gerador de Código (`codegen.py`) é a fase final do compilador. A classe `GeradorYAML` implementa uma variação do padrão Visitor que percorre a AST validada e constrói estruturas de dicionários e listas Python que reproduzem o schema oficial de automações do Home Assistant.

A. Tradução da AST para YAML

1) *Esqueleto de uma automação:* O método `_gerar_automacao` cria o esqueleto base conforme o padrão oficial do Home Assistant:

```
def _gerar_automacao(self, no: dict) -> dict:
    return {
        'id': self._gerar_id(no['nome']),
        'alias': self._limpar_string(no['nome']),
        'description': '',
        'triggers': self._gerar_triggers(no.get('gatilhos', [])),
        'conditions': self._gerar_conditions(no.get('condicoes', [])),
        'actions': self._gerar_actions(no.get('acoes', [])),
        'mode': no.get('modo', 'single'),
    }
```

2) *Geração de IDs*: O compilador gera IDs determinísticos baseados em hash MD5 do nome da automação, garantindo que compilações idênticas produzam IDs idênticos:

```
@staticmethod
def _gerar_id(nome: str) -> str:
    hash_hex = hashlib.md5(nome.encode('utf-8')).hexdigest()
    return str(int(hash_hex[:13], 16))
```

3) *Mapeamento de gatilhos e condições*: Os gatilhos e condições Homi são traduzidos para o schema correspondente do Home Assistant, como descrito nas Tabelas V e VI.

TABLE V
MAPEAMENTO DE GATILHOS PARA SCHEMAS DO HOME ASSISTANT

Tipo Homi	Schema HA
gatilho_evento	trigger: sun
gatilho_estado	trigger: state
gatilho_numerico	trigger: numeric_state
gatilho_horario	trigger: time

TABLE VI
MAPEAMENTO DE CONDIÇÕES PARA SCHEMAS DO HOME ASSISTANT

Tipo Homi	Schema HA
condicao_estado	condition: state
condicao_numerica	condition: numeric_state
condicao_horario	condition: time

4) *Mapeamento de ações*: O gerador extrai o domínio da entidade e formata o serviço correspondente no Home Assistant, com tratamento especial para certos domínios:

```
if a['tipo'] == 'acao_ligar':
    dominio = self._extrair_dominio(a['entidade'])
    if dominio == 'cover':
        action_str = f"{dominio}.open"
    elif dominio == 'automation':
        action_str = f"{dominio}.trigger"
    else:
        action_str = f"{dominio}.turn_on"
```

Para ações de ESPERAR, o tradutor temporal `_parsear_tempo` converte strings simplificadas para o dicionário de delay do HA:

```
# '5min' -> {'hours': 0, 'minutes': 5, 'seconds': 0, 'milliseconds': 0}
# '1h 2min 3s' -> {'hours': 1, 'minutes': 2, 'seconds': 3, 'milliseconds': 0}
```

B. Serialização e indentação no YAML

A exportação final utiliza a biblioteca PyYAML com parâmetros criticamente importantes:

```
def exportar_yaml(self) -> str:
    return yaml.dump(
        self.automacoes_yaml,
        default_flow_style=False,
        sort_keys=False,
        allow_unicode=True,
    )
```

- `sort_keys=False`: preserva a ordem de inserção das chaves (id, alias, triggers, ...), garantindo legibilidade humana.
- `allow_unicode=True`: permite que caracteres acentuados (ex: “Pôr do sol”) sejam salvos diretamente, sem sequências de escape.
- `default_flow_style=False`: força a serialização em estilo bloco (multilinha), mantendo a indentação rígida exigida pelo Home Assistant.

VI. CONCLUSÃO

O compilador Homi demonstra uma implementação completa e funcional das quatro fases clássicas de um compilador: análise léxica (tokenização via AFD/regex), análise sintática (parser LALR(1) com construção de AST), análise semântica (tabela de símbolos e checagem de tipos) e geração de código (serialização YAML). A arquitetura modular permite a extensão independente de cada fase, e o sistema de recuperação de erros em Modo Pânico garante que múltiplos problemas sejam reportados em uma única execução, melhorando significativamente a experiência do usuário final.